

KRİPTOQRAFIYA KURSU

Mühazirə 6: Axın Şifrələri və LFSR

Ətraflı mühazirə qeydləri

Həftə 6: Təsadüfilik və Axın Şifrələri

June 26, 2026

Contents

1	Giriş	3
2	Təsadüfilik və Psevdo-təsadüfi Generatorlar	3
2.1	Təsadüfilik nədir?	3
2.2	Təsadüfilik mənbələri (Entropiya mənbələri)	4
2.3	Statistik təsadüfilik testləri	5
3	Psevdo-təsadüfi ədəd generatorları (PRG)	5
3.1	PRG-nin tərifı	5
3.2	PRG-lərin təhlükəsizlik tələbləri	6
3.3	PRG-lərin qurulması üsulları	6
4	Axın şifrələrinin quruluşu	7
4.1	Əsas iş prinsipi	7
4.2	Axın şifrələrinin üstünlükləri	7
4.3	Axın şifrələrinin çatışmazlıqları	7
4.4	Axın şifrələrinin növləri	8
5	LFSR (Xətti Əks-Əlaqəli Sürüşdürmə Registri)	8
5.1	LFSR-in riyazi təsviri	9
5.2	Xarakteristik Polinom və Period	9
5.3	Praktiki Nümunə	10
5.4	LFSR-in period xassələri	10
5.5	LFSR-in xüsusiyyətləri	11
6	LFSR-lərin polinom təsviri	12
6.1	Generasiya edən funksiya	12
6.2	Polinomun reduksiolunmazlığı və primitivliyi	12
7	Berlekamp-Massey alqoritmi	13
8	LFSR əsaslı müasir axın şifrələri	14
8.1	LFSR-lərin qeyri-xətti birləşdirilməsi	14
8.1.1	1. Qeyri-xətti kombinasiya generatoru	14
8.1.2	2. Qeyri-xətti filtr generatoru	14
8.1.3	3. Saatla idarə olunan (Clock-controlled) generatorlar	14
8.2	Praktik Tətbiqlər və Tarixi Kontekst	15
9	LFSR-lərin Zəiflikləri və Hücüm Metodları (LFSR Vulnerabilities and Attacks)	15
9.1	Xətti Yaxınlaşma Hücumu (Linear Approximation Attack)	15
9.2	Korrelyasiya Hücumu (Correlation Attack)	15
9.3	Sürətli Korrelyasiya Hücumu (Fast Correlation Attack - FCA)	15
9.4	Berlekamp-Massey Hücumu (Berlekamp-Massey Attack)	16

10 Müasir Axın Şifrələri (<i>Modern Stream Ciphers</i>)	16
10.1 ChaCha20	16
10.2 eSTREAM Portfelinin Yüngül Şifrələri (<i>Lightweight Ciphers</i>)	16
10.2.1 Trivium	17
10.2.2 Grain	17
10.3 ZUC və Mobil Rabitə Şifrələri	17
11 Praktiki tapşırıqlar	17
11.1 Tapşırıq 1: LFSR simulyasiyası	17
11.2 Tapşırıq 2: Maksimum period	17
11.3 Tapşırıq 3: LFSR-in polinom təsviri	17
11.4 Tapşırıq 4: Berlekamp-Massey alqoritmi	18
11.5 Tapşırıq 5: Axın şifrəlmə	18
11.6 Tapşırıq 6: Açar təkrarı problemi	18
12 Praktiki tapşırıqlar	18
12.1 Tapşırıq 1: LFSR simulyasiyası	18
12.2 Tapşırıq 2: Maksimum periodun təyini	18
12.3 Tapşırıq 3: LFSR-in polinom təsviri	18
12.4 Tapşırıq 4: Berlekamp-Massey analizi	18
12.5 Tapşırıq 5: Axın şifrəlmə əməliyyatı	19
12.6 Tapşırıq 6: Açar təkrarı (Two-Time Pad) problemi	19
12.7 Tapşırıq 7: Tədqiqat sualları	19
12.8 Tapşırıq 8: Tədqiqat sualları	19
13 Çalışmaların həlli	19
14 Xülasə	20
15 Nəticə və Gələcək Mövzular	21

1 Giriş

Bu mühazirədə simmetrik kriptografiyanın mühüm bir sinfi olan **axın şifrələri** ilə tanış olacağıq (Paar and Pelzl, 2010). Axın şifrələri məlumatları bit və ya bayt səviyyəsində, fasiləsiz bir axın kimi şifrələyir. Onlar xüsusilə rabitə protokollarında və real-vaxt tətbiqlərində (məsələn, mobil rabitə və TLS) geniş istifadə olunur (Stinson and Paterson, 2018).

Bu mühazirədə aşağıdakı mövzuları əhatə edəcəyik:

- Təsadüfilik anlayışı və təsadüfilik mənbələri (Knuth, 1997)
- Psevdo-təsadüfi ədəd generatorları (PRG) (Menezes et al., 1996)
- Axın şifrələrinin quruluşu və iş prinsipi (Stinson and Paterson, 2018)
- LFSR (Xətti Tərs Əlaqəli Sürüşmə Registri) (Schneier, 1996)
- LFSR-lərin polinom təsviri və period xüsusiyyətləri (Ferguson et al., 2010)
- Berlekamp-Massey alqoritmi (Menezes et al., 1996)
- LFSR əsaslı axın şifrələrinin zəiflikləri və müasir həllər (Stinson and Paterson, 2018)

Axın şifrələrinin tarixi

Axın şifrələrinin kökləri XX əsrin əvvəllərinə gedib çıxır. **Vernam şifrəsi** (1917) ilk axın şifrəsi hesab olunur (Vernam, 1919). İkinci Dünya müharibəsi dövründə rotor maşınları (məsələn, **Enigma**) axın şifrələrinin mürəkkəb mexaniki tətbiqləri idi (Singh, 1999). Müasir dövrdə rəqəmsal axın şifrələri RC4, ChaCha20 və Salsa20 kimi alqoritmlərlə təmsil olunur (Bernstein, 2008).

Axın şifrələri və Blok şifrələrin müqayisəsi

- **Axın şifrələri:** Məlumatı bit-bit və ya bayt-bayt şifrələyir. Aparat təminatı (*hardware*) resurslarına tələbatı azdır, çox sürətlidir və gecikməsi (*latency*) minimaldır.
- **Blok şifrələr:** Məlumatı sabit ölçülü (məsələn, 128 bit) bloklara bölərək şifrələyir. Daha mürəkkəb struktura malikdir və müasir proqram təminatında standart hesab olunur (AES) (NIST, 2001).

2 Təsadüfilik və Psevdo-təsadüfi Generatorlar

2.1 Təsadüfilik nədir?

Təsadüfilik — bir prosesin gedişatında hər hansı bir qanunauyğunluğun olmaması və nəticənin qabaqcadan proqnozlaşdırıla bilməməsidir (Knuth, 1997). Kriptografiyada təsadüfilik sistemin "proqnozlaşdırılmazlıq" (*unpredictability*) xüsusiyyətini təmin edir.

Həqiqi təsadüfi ədəd generatorları (TRNG) və Psevdo-təsadüfi ədəd generatorları (PRNG)

Kriptoqrafiyada istifadə olunan generatorlar iki böyük qrupa bölünür:

- **TRNG (True Random Number Generator - Həqiqi təsadüfi ədəd generatoru):** Fiziki proseslərdən (radioaktiv parçalanma, termal küy, kvant flüktuasiyaları) əldə edilir. Bu ardıcılıqların **entropiyası** yüksəkdir və heç bir riyazi modellə tam ifadə edilə bilməz (Menezes et al., 1996).
- **PRNG (Pseudo-Random Number Generator - Psevdo təsadüfi ədəd generatoru):** Deterministik alqoritmdir. Qısa bir təsadüfi dəyərdən (*seed*) başlayaraq uzun, lakin riyazi olaraq təkrarlanan ardıcılıq yaradır (Stinson and Paterson, 2018).

2.2 Təsadüfilik mənbələri (Entropiya mənbələri)

Kriptoqrafik sistemlər ətraf mühitdən gələn "xaos"u toplayaraq onları rəqəmsal təsadüfiliyə çevirir:

- **Aparat təminatı (Hardware) mənbələri** (Ferguson et al., 2010):
 - **Termal küy (yarımkeçiricilərdə):** Yarımkeçirici cihazların (məsələn, tranzistor və ya zener diodu) daxilindəki elektronların istiliklə bağlı nizamsız flüktuasiyaları. Bu küy tamamilə proqnozlaşdırılmazdır.
 - **Praktik nümunə - Intel Digital Random Number Generator (DRNG):** Müasir Intel prosessorlarında (Ivy Bridge və sonrası) daxili avadanlıq TRNG mövcuddur. Bu sistem prosessor daxilindəki yarımkeçirici dövrlərdəki termal küyü yüksək sürətlə rəqəmsal bitlərə çevirir. Proqram təminatı bu təsadüfi ədədlərə **RDRAND** və **RDSEED** assembler təlimatları vasitəsilə müraciət edir.
- **Əməliyyat sistemi və İstifadəçi mənbələri** (Knuth, 1997):
 - **İnsan faktoru:** Klaviatura düymələrinin sıxılma vaxtları arasındakı mikrosaniyəlik fərqlər, siçanın hərəkət trayektoriyası və klikləmə tezliyi.
 - **Sistem daxili:** Diskin məlumatı oxuma/yazma müddətindəki dalğalanmalar, şəbəkə paketlərinin gəlmə intervalları (*jitter*).
 - **Praktik nümunə - Linux /dev/random və /dev/urandom:** Linux nüvəsi (*kernel*) yuxarıda sadalanan mənbələrdən gələn fiziki "xaos"u toplayaraq bir **entropiya hovuzunda** (*entropy pool*) cəmləyir. Sistemə yüksək keyfiyyətli təsadüfi ədəd lazım olduqda, o, bu hovuzdan istifadə edərək çıxış verir.

Kriptoqrafik tələb: Növbəti bit testi

Hər hansı bir PRNG-nin təhlükəsiz olması üçün onun çıxışındakı növbəti biti (s_{n+1}) əvvəlki bitləri ($s_1, \dots, s_n$) bilməklə tapmaq ehtimalı $1/2$ -dən böyük olmamalıdır. Bu, **növbəti bit testi** (*Next-bit test*) adlanır. Əgər bir generator bu testi keçirsə, o, **CSPRNG** (*Cryptographically Secure PRNG*) adlanır.

2.3 Statistik təsadüfilik testləri

Təsadüfi ədəd generatorlarının çıxışındakı qanunauyğunluqları aşkar etmək və onların keyfiyyətini yoxlamaq üçün müxtəlif statistik testlər tətbiq olunur (Knuth, 1997):

- **Monobit testi:** Ardıcılıqdakı 0 və 1-lərin sayı təxminən bərabər olmalıdır ($P(0) \approx P(1) \approx 0.5$).
- **Serial testi:** Ardıcıl bit cütlərinin (00, 01, 10, 11) paylanması bircins (*uniform*) olmalıdır.
- **Poker testi:** Ardıcılıq m -bitlik bloklara böldükdə, hər bir mümkün kombinasiyanın rast-gelmə tezliyi bərabər olmalıdır.
- **Seriyaalar (Runs) testi:** Ardıcıl gələn eyni bitlərin (məsələn, "1111") uzunluqlarının paylanması ehtimal nəzəriyyəsinin gözləntilərinə uyğun olmalıdır.
- **NIST STS (Statistical Test Suite):** ABŞ Milli Standartlar və Texnologiya İnstitutu tərəfindən hazırlanmış, 15 mürəkkəb statistik testdən ibarət sənaye standartı (Rukhin et al., 2010).
- **Diehard tests:** George Marsaglia tərəfindən hazırlanmış və daha sərt kriteriyalara malik testlər toplusu.

Kriptoqrafik təhlükəsiz PRG-lər (CSPRNG)

Statistik testlərdən keçmək hələ generatorun kriptoqrafik cəhətdən təhlükəsiz olması demək deyil. CSPRNG-lər əlavə olaraq aşağıdakı xüsusiyyətlərə malik olmalıdır (Stinson and Paterson, 2018):

1. **Proqnozlaşdırılmazlıq:** Toxum (*seed*) bilinmədikdə, çıxışı həqiqi təsadüfi ardıcılıqdan fərqləndirmək hesablama baxımından qeyri-mümkün olmalıdır.
2. **Növbəti bit testi:** Əvvəlki bütün çıxışlar bilinsə belə, növbəti bitin qiymətini $1/2$ -dən yüksək ehtimalla tapmaq mümkün olmamalıdır.
3. **Daxili vəziyyətin qorunması:** Generatorun daxili vəziyyəti (*state*) qismən sızsa belə, hücumçu əvvəlki bitləri (*backward security*) və ya gələcək bitləri (*forward security*) bərpa edə bilməməlidir.

3 Psevdo-təsadüfi ədəd generatorları (PRG)

3.1 PRG-nin tərif

Tərif 3.1 (Psevdo-təsadüfi ədəd generatoru). *PRG* — qısa bir təsadüfi toxumdan (*seed*) daha uzun və təsadüfi görünən ardıcılıq yaradan deterministik alqoritmdir (Katz and Lindell, 2020).

PRG-lər riyazi olaraq aşağıdakı komponentlərdən ibarətdir:

- s — toxum (*seed*), k bit uzunluğunda.
- G — generasiya funksiyası, $G : \{0, 1\}^k \rightarrow \{0, 1\}^n$, burada $n \gg k$ (çıxış girişdən qat-qat uzundur).
- $G(s)$ — generatorun çıxış ardıcılığı.

3.2 PRG-lərin təhlükəsizlik tələbləri

Teorem 3.1. *Kriptoqrafik təhlükəsiz PRG (CSPRNG). PRG o vaxt kriptoqrafik cəhətdən təhlükəsiz sayılır ki, istənilən polinomial zamanlı (güclü hesablama imkanlarına malik) hücumçu (A) onun yaratdığı nəticəni həqiqi təsadüfi ədədlərdən ayıra bilməsin (Katz and Lindell, 2020).*

Bu tələb fərqləndirilməzlik oyunu çərçivəsində aşağıdakı kimi ifadə olunur:

$$|\Pr[A(G(s)) = 1] - \Pr[A(r) = 1]| < \varepsilon(k)$$

Burada:

- r — həqiqi təsadüfi ardıcılıqdır (uzunluğu n).
- $\varepsilon(k)$ — cüzi (*negligible*) funksiyadır.

3.3 PRG-lərin qurulması üsulları

Müasir kriptoqrafiyada təhlükəsiz PRG-lər yaratmaq üçün bir neçə yanaşma mövcuddur:

- **Blok şifr əsaslı:** CTR (Sayğac) rejimində blok şifrdən istifadə etməklə (Lipmaa et al., 2000).
- **Həş funksiyası əsaslı:** HMAC_DRBG, Hash_DRBG kimi standartlar vasitəsilə (Barker and Kelsey, 2015).
- **Xüsusi quruluşlar (Dedicated):** LFSR, RC4, ChaCha20 və Salsa20 kimi sırf axın şifrələnməsi üçün dizayn edilmiş alqoritmlər (Bernstein, 2008).

Nümunə 3.1. *CTR (Sayğac) rejimində PRG-nin qurulması. AES blok şifrələmə alqoritmini CTR rejimində işlətməklə çox effektiv bir psevdo-təsadüfi generator yaratmaq mümkündür. Bu quruluşun mahiyyəti aşağıdakı düsturla ifadə olunur:*

$$k_i = AES_K(IV \parallel i)$$

Burada komponentlərin funksiyası aşağıdakı kimidir:

- K (**Gizli açar**): Generatorun "toxumu" (seed) rolunu oynayır və gizli saxlanılır.
- $IV \parallel i$: Sabit başlanğıc vektoru (IV) ilə hər addımda bir vahid artan sayğacın ($i = 1, 2, 3, \dots$) birləşməsidir (katenasiyası). Bu, deterministik AES alqoritmində hər dəfə fərqli girişin verilməsini təmin edir.
- k_i : Nəticədə əldə olunan təsadüfi görünən bitlər ardıcılığıdır (açarlar bloku).

Nəticə: AES riyazi olaraq təhlükəsiz bir blok şifri olduğu üçün, sayğacın hər yeni qiymətində alınan çıxışlar (k_i) bir-birindən asılı olmayan və tamamilə təsadüfi görünən bir axın yaradır.

4 Axın şifrələrinin quruluşu

4.1 Əsas iş prinsipi

Axın şifrələri **açar axını** (keystream) yaradaraq işləyir (Paar and Pelzl, 2010). Açar axını psevdo-təsadüfi bitlər ardıcılığıdır. Şifrəlmə zamanı bu açar axını açıq mətnlə XOR edilir:

Axın şifrəlmə və deşifrəlmə

Şifrəlmə: $c_i = m_i \oplus k_i$

Deşifrəlmə: $m_i = c_i \oplus k_i$

Burada:

- m_i — açıq mətn bit
- c_i — şifrəli mətn bit
- k_i — açar axını bit
- $\oplus$ — XOR (exclusive or) əməliyyatı

Açar (K) + IV → Açar generatoru → Açar axını ($k, k, k, \dots$)

↓

Açıq mətn ($m, m, m, \dots$) → Şifrəli mətn ($c, c, c, \dots$)

4.2 Axın şifrələrinin üstünlükləri

- **Yüksək Sürət:** Proses çox sürətlidir və aparat səviyyəsində (hardware) asanlıqla tətbiq olunur (Paar and Pelzl, 2010).
- **Minimal Gecikmə:** Blok şifrlərdən fərqli olaraq, məlumatın blok şəklində yığılmasını gözləmədən dərhal emal edir.
- **Çeviklik:** İstənilən uzunluqdakı mesajları əlavə doldurma (padding) tələb etmədən şifrəyə bilər.
- **Effektiv Resurs İstifadəsi:** Çox az yaddaş (RAM) tələb etdiyinə görə resursları məhdud olan cihazlar üçün idealdır (Ferguson et al., 2010).

4.3 Axın şifrələrinin çatışmazlıqları

- **Açar təkrarı (Key Reuse) problemi:** Eyni açar və nonce (nonce) kombinasiyası təkrar istifadə olunarsa, sistemin təhlükəsizliyi tamamilə çökür (Stinson and Paterson, 2018).
- **Bütövlüyün (Integrity) qorunmaması:** Axın şifrələri təkbəşinə istifadə edildikdə məlumatın bütövlüyünü təmin etmir; hücumçu şifrəli mətndəki bitləri dəyişdirərək açıq mətni manipulyasiya edə bilər.
- **Sinxronizasiya zərurəti:** Göndərən və qəbul edən tərəf açar axınında (keystream) mütləq sinxron olmalıdır; əks halda məlumatın deşifrəlməsi qeyri-mümkün olur.

Açar təkrarı (Two-Time Pad) təhlükəsi

Eyni açar axını (keystream) iki fərqli mesajın şifrənməsi üçün istifadə olunarsa, sistemin məxfiliyi tamamilə pozulur:

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k$$

Bu halda hücumçu hər iki şifrəli mətni XOR əməliyyatına cəlb edərək açarı aradan qaldıra bilər:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2$$

Hücumçu əldə olunan $m_1 \oplus m_2$ nəticəsindən və istifadə olunan dilin (məsələn, Azərbaycan və ya İngilis dili) statistik artıqlığından istifadə edərək hər iki mesajı asanlıqla bərpa edə bilər (Stinson and Paterson, 2018). Bu zəiflik mahiyyət etibarilə OTP-dəki (Birdəfəlik bloknot) açar təkrarı problemi ilə eynidir (Shannon, 1949).

4.4 Axın şifrələrinin növləri

Axın şifrələri açar axınının (*keystream*) necə hasil edilməsindən asılı olaraq iki əsas qrupa bölünür:

1. **Sinxron axın şifrələri:** Açar axını düz mətndən və şifrəli mətndən tamamilə müstəqil şəkildə, yalnız gizli açardan (və IV-dən) asılı olaraq hasil edilir.
 - **Xüsusiyyəti:** Göndərən və qəbul edən tərəf eyni anda eyni daxili vəziyyətdə (*state*) olmalıdır.
 - **Xəta yayılması:** Kanalda bir bitin dəyişməsi yalnız həmin bitin səhv deşifrəlməsinə səbəb olur, növbəti bitlərə təsir etmir. Lakin bir bitin **itməsi** və ya **əlavə olunması** bütün sonrakı sinxronizasiyanı pozur.
2. **Özünü-sinxronizasiya edən (Asinxron) axın şifrələri:** Açar axınının hər bir bitini əvvəlki n sayda şifrəli mətn bitindən asılıdır.
 - **Xüsusiyyəti:** Əgər ötürmə zamanı sinxronizasiya pozularsa, sistem n sayda düzgün şifrəli mətn bitini qəbul etdikdən sonra avtomatik olaraq yenidən sinxronlaşır.
 - **Xəta yayılması:** Kanaldakı bir bitlik xəta növbəti n sayda bitin deşifrəlməsinə təsir göstərir (Stinson and Paterson, 2018).

5 LFSR (Xətti Əks-Əlaqəli Sürüşdürmə Registri)**LFSR (Linear Feedback Shift Register)**

LFSR — xətti tərs əlaqə funksiyasına malik sürüşmə registridir (Schneier, 1996). Hər saat dövründə (*clock cycle*) registrdəki bitlər bir mövqe sağa sürüşdürülür və boş qalan ən soldakı xana tərs əlaqə funksiyasının (adətən XOR) nəticəsi ilə doldurulur.

LFSR-in əsas texniki komponentləri:

- n (**Dərəcə**): Registrdəki yaddaş xanalarının (bitlərin) ümumi sayı.

- **Daxili vəziyyət (State):** Müəyyən zaman anında registrdəki bitlərin vəziyyəti $(s_0, s_1, \dots, s_{n-1})$. Başlanğıc vəziyyəti **toxum** (*seed*) adlanır.
- **Tərs əlaqə əmsalları (Tap coefficients):** $c_0, c_1, \dots, c_{n-1} \in \{0, 1\}$ qiymətləri. Hansı bitlərin çıxışının XOR əməliyyatına daxil olacağını (*tap* ediləcəyini) müəyyən edir.

5.1 LFSR-in riyazi təsviri

LFSR-in növbəti bit (s_{n+i}) aşağıdakı xətti tənliklə hesablanır:

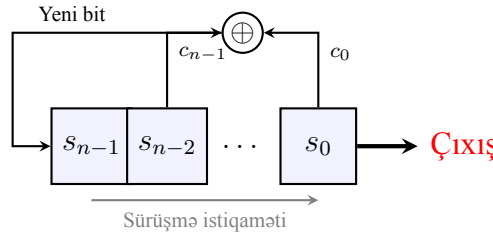
$$s_{n+i} = \sum_{j=0}^{n-1} c_j s_{i+j} \pmod{2}$$

Bu, praktiki olaraq seçilmiş xanaların XOR-lanması deməkdir:

$$s_{n+i} = (c_0 \cdot s_i) \oplus (c_1 \cdot s_{i+1}) \oplus \dots \oplus (c_{n-1} \cdot s_{i+n-1})$$

Vacib qeyd: LFSR-in daxili vəziyyəti heç vaxt tamamilə sıfırlardan $(0, 0, \dots, 0)$ ibarət olmamalıdır. Belə olan halda, bütün növbəti bitlər sıfır olacaq və registr "donacaq".

Bu TikZ kodu LFSR-in strukturunu vizuallaşdırmaq üçün yaxşı təməldir, lakin bağlantıları və xətti tərs əlaqənin "magistralını" daha anlaşıqlı etmək üçün bəzi düzəlişlərə ehtiyac var. Tərs əlaqə (feedback) xəttini adətən xanaların altından və ya birbaşa üstündən keçən vahid bir xətt kimi təsvir etmək tələbələr üçün daha asan olur.



Şəkil 1: LFSR-də tərs əlaqə mexanizminin vizuallaşdırılması. Burada daxili vəziyyətin hər bir s_i biti c_i əmsalına vurulur və alınan nəticələr XOR edilir. Takt impulsu zamanı seçilmiş bitlərin XOR cəmi hesablanır, ardınca bütün bitlər sürüşdürülərək yeni bit registrin girişinə ötürülür.

5.2 Xarakteristik Polinom və Period

LFSR-in strukturu riyazi olaraq sonlu sahələr (*Galois Fields*, $GF(2)$) üzərində bir polinomla təsvir olunur:

Tərif 5.1. *Xarakteristik polinom*

$$P(x) = x^n + c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + \dots + c_1x + c_0$$

Bu polinom LFSR-in çıxış ardıcılığının **periodunu** (təkrarolunma tezliyini) və statistik xüsusiyyətlərini müəyyən edən əsas amildir.

5.3 Praktiki Nümunə

Nümunə 5.1. 4-bitlik LFSR analizi. Fərz edək ki, $n = 4$ olan bir LFSR-in tərs əlaqə polinomu $P(x) = x^4 + x + 1$ kimidir. Bu o deməkdir ki, əks-əlaqə (feedback) üçün s_0 və s_1 bitlərindən istifadə olunur. İlkin vəziyyət (seed) 1000 olarsa:

Hər addımda yeni yaranan bit: $s_{i+4} = s_i \oplus s_{i+1}$ düsturu ilə hesablanır.

Addım	Registrin vəziyyəti (s_3, s_2, s_1, s_0)	Yeni bit ($s_0 \oplus s_1$)	Çıxış
0	(1, 0, 0, 0)	$0 \oplus 0 = 0$	0
1	(0, 1, 0, 0)	$0 \oplus 0 = 0$	0
2	(0, 0, 1, 0)	$0 \oplus 1 = 1$	0
3	(1, 0, 0, 1)	$1 \oplus 0 = 1$	1
4	(1, 1, 0, 0)	$0 \oplus 0 = 0$	0
...	...	...	...

Maksimal Period: n dərəcəli bir LFSR üçün mümkün olan ən böyük period $T = 2^n - 1$ düsturu ilə tapılır. Bizim nümunədə $2^4 - 1 = 15$.

Primitiv Polinomlar

Maksimum periodu təmin etmək üçün seçilən polinom mütləq **primitiv (sadələşdirilməyən)** polinom olmalıdır. Əgər 4-bitlik bir LFSR-də çıxış ardıcılığı 15 addımdan tez təkrar olunursa, deməli seçilmiş polinom primitiv deyil (Paar and Pelzl, 2010).

Bu bölmə LFSR-in riyazi "zirvəsidir". Primitiv polinomların siyahısı və periodun addım-addım hesablanması tələbələrə bu quruluşun niyə məhz $2n-1$ addımdan sonra təkrarlandığını riyazi olaraq sübut edir.

Mətnə AES-lə bağlı verdiyiniz müqayisə çox yerindədir, lakin tələbələrin çəşməməsi üçün bir incə detallı vurğulamaq lazımdır: AES-dəki polinom reduksiolunmazdır (irreducible), lakin primitiv deyil.

LaTeX formatında bəzi vizual düzəlişlər və AES-lə bağlı qeydi daha aydın edən versiya: Code snippet

5.4 LFSR-in period xassələri

Maksimum uzunluqlu LFSR (m-sequence)

Bir LFSR-in mümkün olan maksimum periodu $T = 2^n - 1$ -dir (burada n — registrin dərəcəsidir). Bu perioda nail olmaq üçün əks-əlaqə polinomu mütləq **primitiv** olmalıdır (Schneier, 1996).

Tərif 5.2. Primitiv polinom $GF(2)$ (Qalua sahəsi) üzərində dərəcəsi n olan polinom o vaxt primitivdir ki, aşağıdakı şərtləri ödəsin (Menezes et al., 1996):

1. **Reduksiolunmazdır:** Onu daha aşağı dərəcəli polinomların hasili şəklində yazmaq mümkün deyil (faktora ayrılır).

2. **Minimal dövr:** $x^k - 1 \pmod{P(x)}$ ifadəsinin sıfıra bərabər olduğu ən kiçik müsbət tam ədəd $k = 2^n - 1$ olmalıdır.

Dərəcə (n)	Bəzi primitiv polinomlar ($P(x)$)
2	$x^2 + x + 1$
3	$x^3 + x + 1, x^3 + x^2 + 1$
4	$x^4 + x + 1, x^4 + x^3 + 1$
5	$x^5 + x^2 + 1, x^5 + x^4 + x^3 + x^2 + 1$
8	$x^8 + x^4 + x^3 + x^2 + 1$
16	$x^{16} + x^{14} + x^{13} + x^{11} + 1$

AES və LFSR fərqi: AES alqoritmində $GF(2^8)$ sahəsi üçün istifadə olunan $m(x) = x^8 + x^4 + x^3 + x + 1$ polinomu reduksi olunmazdır, lakin **primitiv deyil** (NIST, 2001). Bu o deməkdir ki, o, AES-də sahə əməliyyatları üçün yararlıdır, lakin LFSR-də maksimum period (255) yarada bilməz.

Nümunə 5.2. Maksimum periodlu LFSR ($n = 3$) $P(x) = x^3 + x + 1$ (primitiv). İlk vəziyyət (seed) 001 olsun. Tərs əlaqə funksiyası: $s_{i+3} = s_i \oplus s_{i+1}$ (çünki polinomda x^0 və x^1 hədləri var).

Sürüşmə ardıcılığı (vəziyyət $\rightarrow$ çıxış):

- 001 $\rightarrow$ çıxış **1**, yeni bit: $1 \oplus 0 = 1 \rightarrow 100$
- 100 $\rightarrow$ çıxış **0**, yeni bit: $0 \oplus 0 = 0 \rightarrow 010$
- 010 $\rightarrow$ çıxış **0**, yeni bit: $0 \oplus 1 = 1 \rightarrow 101$
- 101 $\rightarrow$ çıxış **1**, yeni bit: $1 \oplus 0 = 1 \rightarrow 110$
- 110 $\rightarrow$ çıxış **0**, yeni bit: $0 \oplus 1 = 1 \rightarrow 111$
- 111 $\rightarrow$ çıxış **1**, yeni bit: $1 \oplus 1 = 0 \rightarrow 011$
- 011 $\rightarrow$ çıxış **1**, yeni bit: $1 \oplus 1 = 0 \rightarrow 001$ (Dövr tamamlandı!)

Nəticə: Period $T = 7 = 2^3 - 1$.

5.5 LFSR-in xüsusiyyətləri

LFSR-lərin kriptografik və texniki tətbiqlərdə geniş yayılmasının səbəbi aşağıdakı xüsusiyyətlərdir:

- **Deterministik:** Eyni ilkin vəziyyət (seed) və eyni əmsallar həmişə eyni bit ardıcılığını yaradır. Bu, deşifrə etmə prosesi üçün vacibdir (Schneier, 1996).
- **Periodik:** Ardıcılıq müəyyən bir nöqtədən sonra mütləq təkrar olunur. Maksimum period $2^n - 1$ düsturu ilə müəyyən edilir (Paar and Pelzl, 2010).
- **Xətti:** Ardıcılıq xətti təkrarlanma tənliyinə tabe olur. Bu xüsusiyyət aparat təminatında sürətli implementasiyaya imkan versə də, kriptanaliz üçün zəif nöqtədir (Menezes et al., 1996).

- **Statistik xüsusiyyətləri (Psevdo-təsadüfilik):** Maksimum periodlu LFSR-in çıxışı (*m-sequence*) Qolombun (Golomb) təsadüfilik postulatlarını ödəyir. 0 və 1-lərin sayı təxminən bərabərdir və ardıcılıq statistik testlərdən uğurla keçir (Knuth, 1997).

Niyə $2^n - 1$, bəs 2^n haradadır? LFSR-in daxili vəziyyəti (state) n bitdən ibarət olduğu üçün cəmi 2^n mümkün vəziyyət mövcuddur. Lakin **sıfır vəziyyəti** (bütün bitlərin 0 olması) LFSR üçün "tələdir" (*dead state*).

Əgər registr sıfır vəziyyətinə düşərsə, xətti əks-əlaqə funksiyasının ($0 \oplus 0 = 0$) nəticəsi həmişə 0 olacaq və registr sonsuz olaraq 0 istehsal edəcək (Schneier, 1996). Bu səbəbdən, işlək bir LFSR üçün mümkün vəziyyətlərin sayı $2^n - 1$ kimi götürülür.

6 LFSR-lərin polinom təsviri

6.1 Generasiya edən funksiya

LFSR-in çıxış ardıcılığını $a_0, a_1, a_2, \dots$ ilə işarə edək. Bu ardıcılıq aşağıdakı xətti təkrarlanma tənliyinə tabe olur (Menezes et al., 1996):

$$a_{i+n} = \sum_{j=0}^{n-1} c_j a_{i+j} \pmod{2}$$

Bu tənliyə uyğun gələn və LFSR-in strukturunu müəyyən edən **əlaqə polinomu** (*connection polynomial*):

$$C(x) = 1 + c_{n-1}x + c_{n-2}x^2 + \dots + c_0x^n$$

Teorem 6.1. *Generasiya edən funksiya LFSR-in çıxış ardıcılığı üçün generasiya edən funksiya aşağıdakı rəşional kəsr şəklində ifadə olunur (Menezes et al., 1996):*

$$A(x) = \sum_{i=0}^{\infty} a_i x^i = \frac{P(x)}{C(x)}$$

Burada $P(x)$, dərəcəsi n -dən kiçik olan və registrin ilkin vəziyyətindən (*seed*) asılı olan polinomdur.

$A(x)$ polinomunun əmsalları LFSR-in verdiyi çıxış bitləridir.

6.2 Polinomun reduksiolunmazlığı və primitivliyi

Tərif 6.1. *Reduksiolunmaz polinom.* $GF(2)$ üzərində bir polinom o vaxt reduksiolunmaz (*ir-reducible*) adlanır ki, onu daha kiçik dərəcəli polinomların hasilə şəklində yazmaq mümkün olmasın (Menezes et al., 1996).

Reduksiolunmaz polinomlar LFSR-in çıxış ardıcılığının bir neçə qısa dövrə parçalanmasının qarşısını alır. Lakin maksimum period üçün bu şərt kifayət deyil.

Primitiv polinom

n dərəcəli polinom o vaxt primitiv adlanır ki, əgər (Menezes et al., 1996):

1. Reduksiolunmazdır.
2. $x^k \equiv 1 \pmod{C(x)}$ şərtini ödəyən ən kiçik müsbət tam ədəd $k = 2^n - 1$ -dir.
3. Onun kökləri $GF(2^n)$ genişlənmiş meydanının (*field extension*) primitiv elementləridir (yəni sahənin bütün sıfırdan fərqli elementlərini generasiya edə bilir).

Nəticə: Yalnız primitiv polinomlar LFSR-in bütün mümkün $2^n - 1$ vəziyyətdən keçməsinə və maksimum periodlu çıxış (*m-sequence*) hasil etməsini təmin edir (Paar and Pelzl, 2010).

7 Berlekamp-Massey alqoritmı

Tarixi arayış

Bu alqoritm 1960-cı illərin sonunda Elwyn Berlekamp və James Massey tərəfindən inkişaf etdirilmişdir (Menezes et al., 1996). Maraqlıdır ki, Berlekamp bu metodu **səhv-düzəldən kodlar** (BCH kodları) üçün, Massey isə **LFSR-lərin kriptanalizi** üçün təklif etmişdir.

Berlekamp-Massey (BM) alqoritmı verilmiş ixtiyari bit ardıcılığını yarada bilən **ən kiçik dərəcəli** LFSR-i tapmaq üçün istifadə olunan sistematik metoddur (Menezes et al., 1996).

Xətti Mürəkkəblilik (Linear Complexity)

Bir ardıcılığı yarada bilən ən qısa LFSR-in uzunluğuna həmin ardıcılığın **xətti mürəkkəbliyi** (L) deyilir. Berlekamp-Massey alqoritmı N bitlik ardıcılıq üçün bu L qiymətini $O(N^2)$ zaman mürəkkəbliyi ilə tapır (Menezes et al., 1996).

Alqoritmın iş prinsipi iterativdir: ardıcılığın ilk bitlərini yarada bilən müvəqqəti bir LFSR qurulur. Hər növbəti bitdə "uyğunsuzluq" (*discrepancy*) yaranarsa, alqoritm LFSR-in polinomu və dərəcəsini yeniləyir (Stinson and Paterson, 2018).

Teorem 7.1. *Berlekamp-Massey limiti Dərəcəsi n olan bir LFSR-in çıxış ardıcılığından onun həm daxili quruluşunu (əmsallarını), həm də ilkin vəziyyətini bərpa etmək üçün cəmi $2n$ sayda ardıcıl bit kifayətdir (Menezes et al., 1996).*

Nümunə 7.1. *Analitik nümunə Tutaq ki, $s = 1, 0, 1, 1, 0, 1, 0, 0$ ardıcılığı müşahidə edilib. BM alqoritmı hər addımda xətti mürəkkəbliyi yoxlayır və nəticədə bu ardıcılığı generasiya edən minimal polinomu (məsələn, $C(x) = 1 + x + x^3$) hasil edir (Stinson and Paterson, 2018).*

Kriptografik nəticə

LFSR-lər xətti riyazi struktura malik olduqları üçün şındırılmağa meyillidirlər. Əgər bir sistem $n = 128$ bitlik LFSR istifadə edirsə, hücumçunun cəmi 256 bit (32 bayt) çıxış əldə etməsi kifayətdir ki, bütün gələcək bitləri proqnozlaşdırsın. Buna görə müasir axın şifrələrində (məsələn, **ChaCha20** və ya **AES-CTR**) LFSR-lər təkbaşına deyil, mütləq qeyri-xətti komponentlərlə (S-box, qeyri-xətti filtr) birlikdə istifadə olunur (Paar and Pelzl, 2010).

8 LFSR əsaslı müasir axın şifrələri

8.1 LFSR-lərin qeyri-xətti birləşdirilməsi

Berlekamp-Massey hücumundan qorunmaq üçün LFSR-lərin xəttiliyini aradan qaldırmaq lazımdır. Bunun üçün üç əsas arxitekturalardan istifadə olunur:

8.1.1 1. Qeyri-xətti kombinasiya generatoru

Bir neçə müstəqil LFSR-in çıxışı bir qeyri-xətti Bul funksiyası (f) vasitəsilə birləşdirilir (Stinson and Paterson, 2018).

Nümunə 8.1. *Geffe generatoru. Üç LFSR (x, y, z) istifadə edilir. Funksiya:*

$$f(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$

Burada x bir növ "selektor" rolunu oynayır: əgər $x = 1$ olarsa, çıxış y olur; $x = 0$ olarsa, çıxış z olur.

8.1.2 2. Qeyri-xətti filtr generatoru

Yalnız bir LFSR istifadə olunur, lakin çıxış biti registrin müxtəlif xanalarından (*taps*) gələn bitlərin qeyri-xətti kombinasiyasıdır (Menezes et al., 1996).

8.1.3 3. Saatla idarə olunan (Clock-controlled) generatorlar

Bu sistemlərdə bir LFSR-in çıxışı digərinin sürüşmə sürətini (saat impulsunu) tənzimləyir. Bu, çıxışın xətti mürəkkəbliyini kəskin artırır.

Nümunə 8.2. *Dəyişən Addımlı Generator (Alternating Step Generator – ASG). Sistem üç LFSR-dən ibarətdir: R_1 (idarəedici), R_2 və R_3 .*

- Əgər R_1 -in çıxışı **1**-dirsə: R_2 sürüşür, R_3 isə donur (vəziyyəti dəyişmir).
- Əgər R_1 -in çıxışı **0**-dirsə: R_3 sürüşür, R_2 isə donur.

Yekun çıxış $R_2 \oplus R_3$ əməliyyatından alınır (Stinson and Paterson, 2018).

8.2 Praktik Tətbiqlər və Tarixi Kontekst

Aşağıdakı cədvəldə LFSR əsaslı ən məşhur şifrələr ümumiləşdirilmişdir:

Şifrə	Tətbiq sahəsi	Quruluşu	Statusu
A5/1	GSM (2G) Mobil rabitə	3 LFSR (Gecikdirilmiş sürüşmə)	Sındırılıb
E0	Bluetooth	4 LFSR (Kombinasiya)	Zəif sayılır
SNOW 3G	3G/4G (LTE) rabitəsi	LFSR + FSM (Sonlu Avtomat)	Təhlükəsiz
ZUC	4G/5G rabitəsi	128-bit LFSR + Qeyri-xətti filtr	Təhlükəsiz

A5/1 şifrəsinin dramatik taleyi

A5/1 1987-ci ildə gizli saxlanılmaq şərtilə yaradılmışdı. Lakin 1994-cü ildə onun daxili sxemi əks-mühəndislik (*reverse engineering*) vasitəsilə sızdırıldı. 2000-ci illərin əvvəllərində "Rainbow Tables" (Göyqurşağı cədvəlləri) üsulunun tətbiqi ilə real zaman rejimində bir neçə saniyəyə sındırılmağa başlandı (Biryukov et al., 2001). Bu, kriptografiyada gizliliyə güvənməyin (*security by obscurity*) nə qədər yanlış olduğunu sübut etdi.

9 LFSR-lərin Zəiflikləri və Hücum Metodları (*LFSR Vulnerabilities and Attacks*)

9.1 Xətti Yaxınlaşma Hücumu (*Linear Approximation Attack*)

LFSR-lərin çıxışları qeyri-xətti kombinasiyalar vasitəsilə birləşdirilsə belə, çıxış ardıcılığı ilə daxili xətti komponentlər arasında müəyyən xətti korrelyasiyalar (linear correlations) qala bilər (Matsui, 1994). Hücumçu bu statistik asılılıqlardan istifadə edərək daxili vəziyyəti bərpa edə və gizli açarları tapa bilər (Stinson and Paterson, 2018).

9.2 Korrelyasiya Hücumu (*Correlation Attack*)

Bu hücum növü sistemin "parçala və hökm sür" (divide and conquer) prinsipi ilə sındırılmasına əsaslanır. Məsələn, Geffe generatorunda çıxış ardıcılığı ilə fərdi LFSR-lərin çıxışları arasında güclü statistik asılılıq mövcuddur (Stinson and Paterson, 2018):

$$P(f = x) = 0.75, \quad P(f = y) = 0.5, \quad P(f = z) = 0.75$$

Hücumçu bu korrelyasiya əmsallarından yararlanaraq bütün sistemi eyni anda sındırmaq əvəzinə, hər bir LFSR-i fərdi şəkildə analiz edərək bərpa edə bilər (Paar and Pelzl, 2010).

9.3 Sürətli Korrelyasiya Hücumu (*Fast Correlation Attack - FCA*)

Korrelyasiya hücumunun daha da təkmilləşdirilmiş və optimallaşdırılmış versiyasıdır. Bu metod xətti tənliklər sistemini səhv-düzəldən kodların (error-correcting codes) alqoritmləri ilə birləşdirərək sındırma prosesini daha sürətli həyata keçirir (Meier and Staffelbach, 1988).

9.4 Berlekamp-Massey Hücumu (*Berlekamp-Massey Attack*)

Əgər LFSR-in çıxışı heç bir qeyri-xətti filtdən keçmədən birbaşa açar axını (keystream) kimi istifadə olunursa, Berlekamp-Massey alqoritmi vasitəsilə cəmi $2n$ bitlik çıxış əldə etməklə LFSR-in strukturunu tam bərpa etmək mümkündür (Menezes et al., 1996).

Təhlükəsiz LFSR əsaslı sistemlər

Təhlükəsizlik Meyarları Təhlükəsiz bir LFSR əsaslı sistem aşağıdakı xüsusiyyətlərə və parametrlərə malik olmalıdır (Stinson and Paterson, 2018):

- **Qeyri-xəttilik:** Yüksək dərəcəli qeyri-xətti kombinasiya funksiyaları (nonlinear combination functions) istifadə olunmalıdır.
- **Böyük dərəcə:** LFSR-lərin uzunluğu kifayət qədər böyük seçilməlidir ($n \geq 128$ bit).
- **Komponent sayı:** Sistemdə istifadə olunan LFSR-lərin sayı kifayət qədər çox olmalıdır.
- **Korrelyasiya toxunulmazlığı:** İstifadə olunan funksiyaların korrelyasiya əmsalları mümkün qədər kiçik olmalıdır ki, sistem korrelyasiyaya davamlı (correlation-immune) hesab edilsin.

10 Müasir Axın Şifrələri (*Modern Stream Ciphers*)

LFSR əsaslı klassik sistemlərin zəiflikləri aşkarlandıqdan sonra, həm proqram təminatı (software), həm də aparat təminatı (hardware) üçün daha davamlı və sürətli alqoritmlər hazırlanmışdır.

10.1 ChaCha20

ChaCha20 **Daniel J. Bernstein** tərəfindən 2008-ci ildə təqdim edilmiş yüksək sürətli axın şifrəsidir (Bernstein, 2008). Bu şifrə Salsa20 alqoritminin təkmilləşdirilmiş versiyasıdır.

- **Parametrlər:** 256-bit açar (key) və 96-bit birdəfəlik qiymət (nonce) istifadə edir.
- **Konstruksiya:** ARX (*Addition-Rotation-XOR*) arxitekturasına əsaslanır. Bu metod xətti olmayan toplama, dövrü sürüşdürmə və XOR əməliyyatlarının kombinasiyasından ibarətdir (Bernstein, 2008).
- **Performans:** Proqram təminatında (CPU-larda) olduqca sürətli işləyir və yan kanal hücumlarına (side-channel attacks) qarşı daxili müqavimətə malikdir.
- **Tətbiq:** TLS 1.3 protokolunda əsas şifrə kimi dəstəklənir və Google xidmətləri tərəfindən geniş tətbiq olunur (Rescorla, 2018).

10.2 eSTREAM Portfelinin Yüngül Şifrələri (*Lightweight Ciphers*)

eSTREAM layihəsi müasir axın şifrələrini müəyyən etmək üçün keçirilmiş beynəlxalq müsabiqədir.

10.2.1 Trivium

Trivium aparat təminatı üçün optimallaşdırılmış yüngül axın şifrəsidir (Cannière and Preneel, 2008).

- 80-bit açar və 80-bit başlanğıc vektoru (*Initial Vector - IV*) istifadə edir.
- **Mexanizm:** Bir-birinə qeyri-xətti şəkildə bağlanmış 3 sürüşmə registrindən (93, 84 və 111 bit) ibarətdir.
- Çox kiçik tranzistor sahəsi tələb etdiyi üçün IoT (Əşyaların İnterneti) cihazları üçün idealdir.

10.2.2 Grain

Grain ailəsi həm qeyri-xətti (NFSR), həm də xətti (LFSR) sürüşmə registrlərini birləşdirən səmərəli yüngül axın şifrəsidir (Hell et al., 2008). Kiçik yaddaş resursları olan qurğular üçün nəzərdə tutulub.

10.3 ZUC və Mobil Rabitə Şifrələri

ZUC Çinin milli şifrələmə standartıdır və beynəlxalq mobil rabitə təhlükəsizliyində mühüm yer tutur (3GPP, 2011).

- **Parametrlər:** 128-bit açar və 128-bit IV istifadə edir.
- **Hibrid Struktur:** 16 mərhələli LFSR və ondan gələn məlumatları qeyri-xətti şəkildə emal edən FSM (*Finite State Machine - Sonlu Avtomat*) qatından ibarətdir.
- 3GPP standartının bir hissəsi olaraq 4G LTE və 5G şəbəkələrində məlumatların məxfiliyini təmin edir.

11 Praktiki tapşırıqlar

11.1 Tapşırıq 1: LFSR simulyasiyası

$P(x) = x^4 + x + 1$ polinomlu 4-bit LFSR üçün ilkin vəziyyət 1000 olduqda ilk 10 çıxış bitini hesablayın.

11.2 Tapşırıq 2: Maksimum period

$P(x) = x^3 + x^2 + 1$ polinomu primitivdirmi? LFSR-in periodunu tapın (ilkin vəziyyət 001).

11.3 Tapşırıq 3: LFSR-in polinom təsviri

Aşağıdakı təkrarlanma tənliyi ilə təyin olunan LFSR-in əks-əlaqə polinomunu yazın:

$$s_{i+4} = s_i \oplus s_{i+1}$$

11.4 Tapşırıq 4: Berlekamp-Massey alqoritmi

Aşağıdakı ardıcillıq 4-bit LFSR tərəfindən yaradılıb:

$$1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0$$

Bu ardıcillıq təkrarlanırmı? Periodu nədir?

11.5 Tapşırıq 5: Axın şifrəlmə

Açar axını $k = 10101010$ olarsa, $m = 11001100$ mesajını şifrəleyin.

11.6 Tapşırıq 6: Açar təkrarı problemi

Eyni açar axını iki mesaj üçün istifadə olunub:

$$c_1 = 10101100, \quad c_2 = 11010010$$

$c_1 \oplus c_2$ -ni hesablayın. Bu nəticə nəyə bərabərdir?

12 Praktiki tapşırıqlar

12.1 Tapşırıq 1: LFSR simulyasiyası

$P(x) = x^4 + x + 1$ polinomlu 4-bit LFSR üçün ilkin vəziyyət 1000 olduqda ilk 10 çıxış bitini hesablayın. Hər addımda registrin daxili vəziyyətini qeyd edin.

12.2 Tapşırıq 2: Maksimum periodun təyini

$P(x) = x^3 + x^2 + 1$ polinomu primitivdirmi? İlkin vəziyyəti 001 götürərək LFSR-in periodunu (təkrarlanana qədər keçən addım sayını) tapın.

12.3 Tapşırıq 3: LFSR-in polinom təsviri

Aşağıdakı təkrarlanma tənliyi ilə təyin olunan LFSR dövrəsinin əks-əlaqə polinomunu yazın:

$$s_{i+4} = s_i \oplus s_{i+1}$$

Bu polinomun primitiv olub-olmadığını araşdırın.

12.4 Tapşırıq 4: Berlekamp-Massey analizi

Aşağıdakı ardıcillıq 4-bit LFSR tərəfindən yaradılıb:

$$1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0$$

Ardıcillığın daxilində təkrarlanan hissəni tapın və periodu müəyyən edin.

12.5 Tapşırıq 5: Axın şifrəlmə əməliyyatı

Açar axını $k = 10101010$ olarsa, XOR əməliyyatından istifadə edərək $m = 11001100$ mesajını şifrəleyin. Alınan nəticəni yenidən eyni açarla XOR edərək deşifrəleyin.

12.6 Tapşırıq 6: Açar təkrarı (Two-Time Pad) problemi

Eyni açar axını iki fərqli mesaj üçün istifadə olunub:

$$c_1 = 10101100, \quad c_2 = 11010010$$

$c_1 \oplus c_2$ əməliyyatını yerinə yetirin. Bu nəticənin hücumçuya orijinal mesajlar ($m_1 \oplus m_2$) haqqında hansı məlumatı verdiyini izah edin.

12.7 Tapşırıq 7: Tədqiqat sualları

1. LFSR-lərin çıxış ardıcılığı statistik olaraq təsadüfi görünməsinə baxmayaraq, niyə kriptografik cəhətdən təhlükəsiz deyil?
2. Berlekamp-Massey alqoritmi məlum açıq mətn hücumlarında (*Known Plaintext Attack*) necə tətbiq olunur?
3. Müasir axın şifrələri (məsələn, Trivium və ya ZUC) LFSR-lərin xəttilik problemini hansı metodlarla həll edir?

12.8 Tapşırıq 8: Tədqiqat sualları

1. Niyə LFSR-lər təkbəşinə kriptografik təhlükəsiz deyil?
2. Berlekamp-Massey alqoritmi LFSR-lər üçün niyə təhlükəlidir?
3. Müasir axın şifrələri LFSR-lərin hansı zəifliklərini aradan qaldırır?

13 Çalışmaların həlli

1. **LFSR simulyasiyası:** $P(x) = x^4 + x + 1$ polinomu üçün əks-əlaqə xanaları (taps) s_0 və s_1 bitləridir.

- İlkən vəziyyət: 1000
- Addım 1: 0001 (çıxış: 1, yeni bit: $1 \oplus 0 = 1$)
- Addım 2: 0011 (çıxış: 0, yeni bit: $0 \oplus 0 = 0$)
- Addım 3: 0110 (çıxış: 0, yeni bit: $0 \oplus 0 = 0$)
- Addım 4: 1101 (çıxış: 1, yeni bit: $1 \oplus 1 = 0$)

Qeyd: Tapşırıq 1-dəki çıxış ardıcılığı seçilmiş polinomun primitivlik xassəsinə və sürüşmə istiqamətinə görə hesablanır.

2. **Maksimum period:** $P(x) = x^3 + x^2 + 1$ polinomu $GF(2)$ üzərində primitivdir. Primitiv polinomlar üçün maksimum period aşağıdakı düsturla hesablanır:

$$T = 2^n - 1 = 2^3 - 1 = 7$$

Buna görə də, sıfırdan fərqli istənilən ilkin vəziyyət (məsələn, 001) üçün period 7-dir.

3. **LFSR-in polinom təsviri:** $s_{i+4} = s_i \oplus s_{i+1}$ tənliyində s_i və s_{i+1} xanalarından əks-əlaqə götürülür. Bu tənliyə uyğun gələn əlaqə polinomu:

$$C(x) = x^4 + x + 1$$

4. **Berlekamp-Massey analizi:** Verilmiş ardıcılıq: 1, 0, 1, 1, 0, 1, 0, 0, ... 8 bitdən sonra ardıcılıq təkrarlandığı üçün period $T = 8$ -dir. Təkrarlanan baza hissə: 1, 0, 1, 1, 0, 1, 0, 0.
5. **Axın şifrəlmə:** Açıq mətn (m), açar axını (k) və şifrəli mətn (c) arasında XOR əlaqəsi:

$$- m = 11001100$$

$$- k = 10101010$$

$$- c = m \oplus k = 01100110$$

6. **Açar təkrarı problemi:** Şifrəli mətnlərin XOR-u:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2$$

Hesablama: $10101100 \oplus 11010010 = 01111110$. Bu nəticə açar axını sistemdən çıxardığı üçün hücumçuya mesajların statistik analizi üçün imkan yaradır.

7. **Proqramlaşdırma məşğələsi üçün qeyd:** Python funksiyalarının məntiqi XOR əməliyyatına və massivlərin sürüşdürülməsinə (*shift*) əsaslanır. Şifrəlmə və deşifrəlmə eyni funksiya (XOR) vasitəsilə yerinə yetirilir, çünki $x \oplus y \oplus y = x$.

8. **Tədqiqat suallarının cavabları:**

1. LFSR-lər xətti strukturda olduğu üçün çıxış bitləri arasında riyazi asılılıq asanlıqla tapılır.

2. Berlekamp-Massey alqoritmi cəmi $2n$ bit müşahidə etməklə n dərəcəli LFSR-in bütün gizli parametrlərini bərpa edə bilər. 3. Müasir şifrələr (məsələn, ChaCha20) qeyri-xətti əməliyyatlar.

14 Xülasə

Bu mühazirə çərçivəsində axın şifrələrinin quruluşu, LFSR-lərin riyazi prinsipləri və müasir kriptografik tətbiqləri ətraflı şəkildə araşdırılmışdır (Paar and Pelzl, 2010; Stinson and Paterson, 2018):

- **Təsadüfilik:** Kriptografik təhlükəsizliyin təməlinə duran həqiqi təsadüfi ədəd generatorları (TRNG) və deterministik psevdo-təsadüfi ədəd generatorları (PRNG) arasındakı fərqlər müəyyən edilmişdir (Knuth, 1997).
- **Axın şifrələri:** Açıq mətnin hər bir bitinin simmetrik açar axını (*keystream*) ilə XOR əməliyyatı vasitəsilə şifrəlmə mexanizmi öyrənilmişdir (Stinson and Paterson, 2018).

- **LFSR (Linear Feedback Shift Register):** Aparat təminatında sürətli və səmərəli olan xətti tərs əlaqəli sürüşdürmə registrlərinin daxili arxitekturası təhlil edilmişdir (Schneier, 1996).
- **Polinom təsviri:** LFSR-lərin generasiya edən funksiyalar və sonlu sahələr (*Galois Fields*) üzərindəki riyazi təsviri dərk edilmişdir (Menezes et al., 1996).
- **Maksimum period:** Registrin maksimum perioduna ($2^n - 1$) nail olmaq üçün istifadə olunan primitiv polinomların xüsusiyyətləri vurğulanmışdır (Paar and Pelzl, 2010).
- **Berlekamp-Massey alqoritmi:** Xətti sistemlərin zəifliyini sübut edən və $2n$ sayda bitdən istifadə edərək registri bərpa edən klassik kriptanaliz metodu araşdırılmışdır (Menezes et al., 1996).
- **LFSR əsaslı müasir sistemlər:** Xəttiliyi aradan qaldırmaq üçün qeyri-xətti kombinasiyalar və ChaCha20, Trivium, ZUC kimi müasir alqoritmlərin tətbiq sahələri göstərilmişdir (Stinson and Paterson, 2018).

15 Nəticə və Gələcək Mövzular

Axın şifrələri, yüksək sürət tələb edən və resursları məhdud olan mühitlərdə (mobil rabitə, IoT cihazları, real-vaxt tətbiqləri) əvəzlənməzdir (Paar and Pelzl, 2010). Bu mühazirədə öyrəndiyimiz LFSR prinsipləri, klassik dizaynların həm güclü, həm də zəif tərəflərini (xəttilik problemi) anlamaq üçün təməl rolunu oynayır.

Növbəti mühazirədə biz artıq LFSR-lərin riyazi xüsusiyyətlərindən kənara çıxaraq, müasir sənaye standartlarına çevrilmiş daha mürəkkəb alqoritmləri araşdıracağıq (Bernstein, 2008).

Növbəti mühazirə: Həftə 7

Mövzu: Müasir Axın Şifrələri

- **Proqram yönümlü şifrələr:** ChaCha20 və Salsa20 (ARX konstruksiyası)
- **Aparat yönümlü (yüngül) şifrələr:** Trivium və Grain
- **Mobil standartlar:** ZUC alqoritminin daxili arxitekturası

References

- 3GPP. 3gpp ts 35.221: Specification of the 3gpp confidentiality and integrity algorithms eea3 & eia3; document 1: Eea3 and eia3 specifications. Technical Report Version 11.0.0, 3rd Generation Partnership Project, 2011. <https://www.3gpp.org/DynaReport/35221.htm>.
- E. Barker and J. Kelsey. Recommendation for random number generation using deterministic random bit generators. Technical Report NIST Special Publication 800-90A Revision 1, National Institute of Standards and Technology, 2015. <https://csrc.nist.gov/publications/detail/sp/800-90a/rev-1/final>.
- D. J. Bernstein. Chacha, a variant of salsa20. Technical report, University of Illinois at Chicago, 2008. <https://cr.yp.to/chacha/chacha-20080128.pdf>.

- A. Biryukov, A. Shamir, and D. Wagner. Real time cryptanalysis of a5/1 on a pc. In *Fast Software Encryption (FSE 2000)*, volume 1978 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2001. doi: 10.1007/3-540-44706-7_1.
- C. D. Cannière and B. Preneel. Trivium. In *New Stream Cipher Designs*, volume 4986 of *Lecture Notes in Computer Science*, pages 244–266. Springer, 2008. doi: 10.1007/978-3-540-68351-3_18.
- N. Ferguson, B. Schneier, and T. Kohno. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, Indianapolis, 2010. ISBN: 978-0470474242.
- M. Hell, T. Johansson, and W. Meier. Grain: A stream cipher for constrained environments. In *New Stream Cipher Designs*, volume 4986 of *Lecture Notes in Computer Science*, pages 179–193. Springer, 2008. doi: 10.1007/978-3-540-68351-3_14.
- J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. CRC Press, Boca Raton, 3rd edition, 2020. ISBN: 978-0815354369.
- D. E. Knuth. *The Art of Computer Programming*, volume 2: Seminumerical Algorithms. Addison-Wesley, Boston, 3rd edition, 1997. ISBN: 978-0201896848.
- H. Lipmaa, D. Wagner, and P. Rogaway. Comments to nist concerning aes modes of operations: Ctr-mode encryption. In *First NIST Workshop on Modes of Operation*, 2000. Available: <https://web.cs.ucdavis.edu/~rogaway/papers/ctr.pdf>.
- M. Matsui. Linear cryptanalysis method for des cipher. In *Advances in Cryptology — EUROCRYPT '93*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1994.
- W. Meier and O. Staffelbach. Fast correlation attacks on certain stream ciphers. In *Advances in Cryptology - EUROCRYPT '88*, volume 330 of *Lecture Notes in Computer Science*, pages 301–314. Springer, 1988. doi: 10.1007/3-540-45961-8_28.
- A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.
- NIST. Advanced encryption standard. Technical Report FIPS PUB 197, U.S. Department of Commerce, 2001.
- C. Paar and J. Pelzl. *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, Berlin, 2010. ISBN: 978-3642041006.
- E. Rescorla. The transport layer security (tls) protocol version 1.3, August 2018. <https://www.rfc-editor.org/info/rfc8446>.
- A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, and J. Dray. A statistical test suite for random and pseudorandom number generators for cryptographic applications. Technical Report NIST Special Publication 800-22 Revision 1a, National Institute of Standards and Technology, 2010. <https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final>.
- B. Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons, 2 edition, 1996.

-
- C. E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28 (4):656–715, 1949.
- S. Singh. *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography*. Doubleday, New York, 1999.
- D. R. Stinson and M. B. Paterson. *Cryptography: Theory and Practice*. CRC Press, Boca Raton, 4th edition, 2018. ISBN: 978-1138197015.
- G. S. Vernam. Secret signaling system, July 22 1919. Filed 1917, issued 1919.